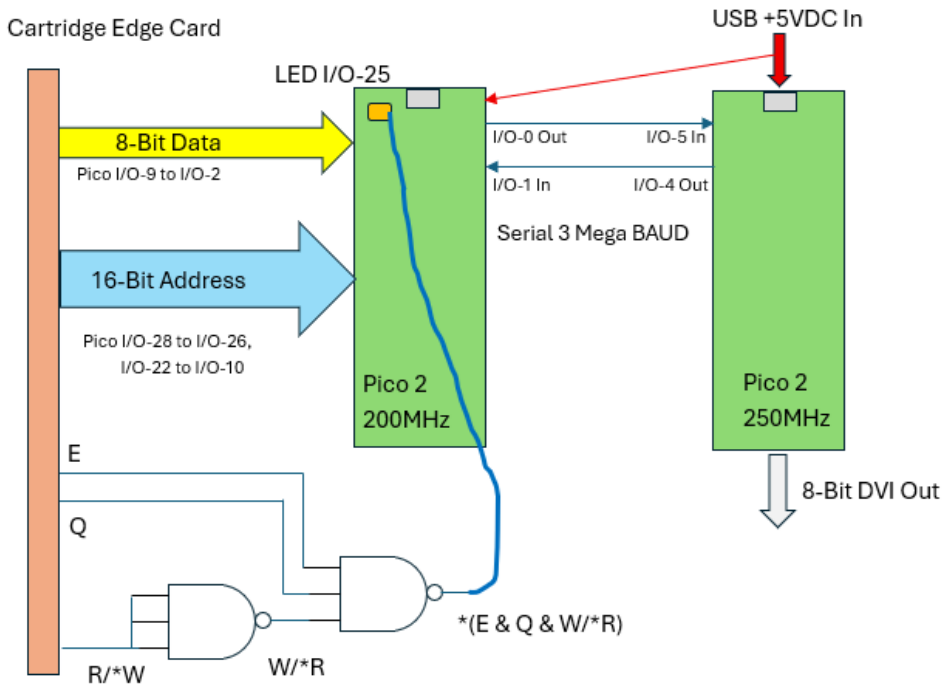
Tandy Color Computer II DVI Build Simple-Circuit 2026

I recently purchased and repaired an old Tandy Color Computer II. It was originally designed to produce video through an RF modulator. I had only one monitor that could receive NTSC broadcast signals. Not wanting to modify the internals of the computer, I wondered if internal data writes could be captured through the cartridge interface. Writes to video memory would then be processed into VGA DVI video for modern computer monitors.

One of my previous projects was a VGA serial text monitor interface using the Raspberry Pi Pico 2. The Pico 2 uses 8 I/O pins for video generation, and a second Pico would be needed to monitor the computer bus. The second Pico would also require bidirectional communication with the Pico generating the video signal.

The Color Computer has 16 address lines, 8 data lines, a two-phase clock, and one read/write line. The clocks and read/write signals can be logically ANDed together to generate a pulse every time a write to memory occurs. In addition to the 25 bus signals, two additional I/O pins are needed for the processor-to-processor communication. The Pico only has 26 available I/O pins. One extra I/O can be had by removing the I/O-25 LED and using it as an input.



For communication between the processors, UART serial I/O is used. The maximum BAUD clock I was able to achieve was 3MHz. The video memory is 6144 bytes maximum. One start bit, one stop bit, eight data bits and 6144 bytes equals 20.5ms per transfer. The refresh can be up to 48Hz which would be acceptable. However, display update will eat up some extra time. A max time out of 50ms per refresh was used to prevent potential lockup if waiting for a missing handshake.

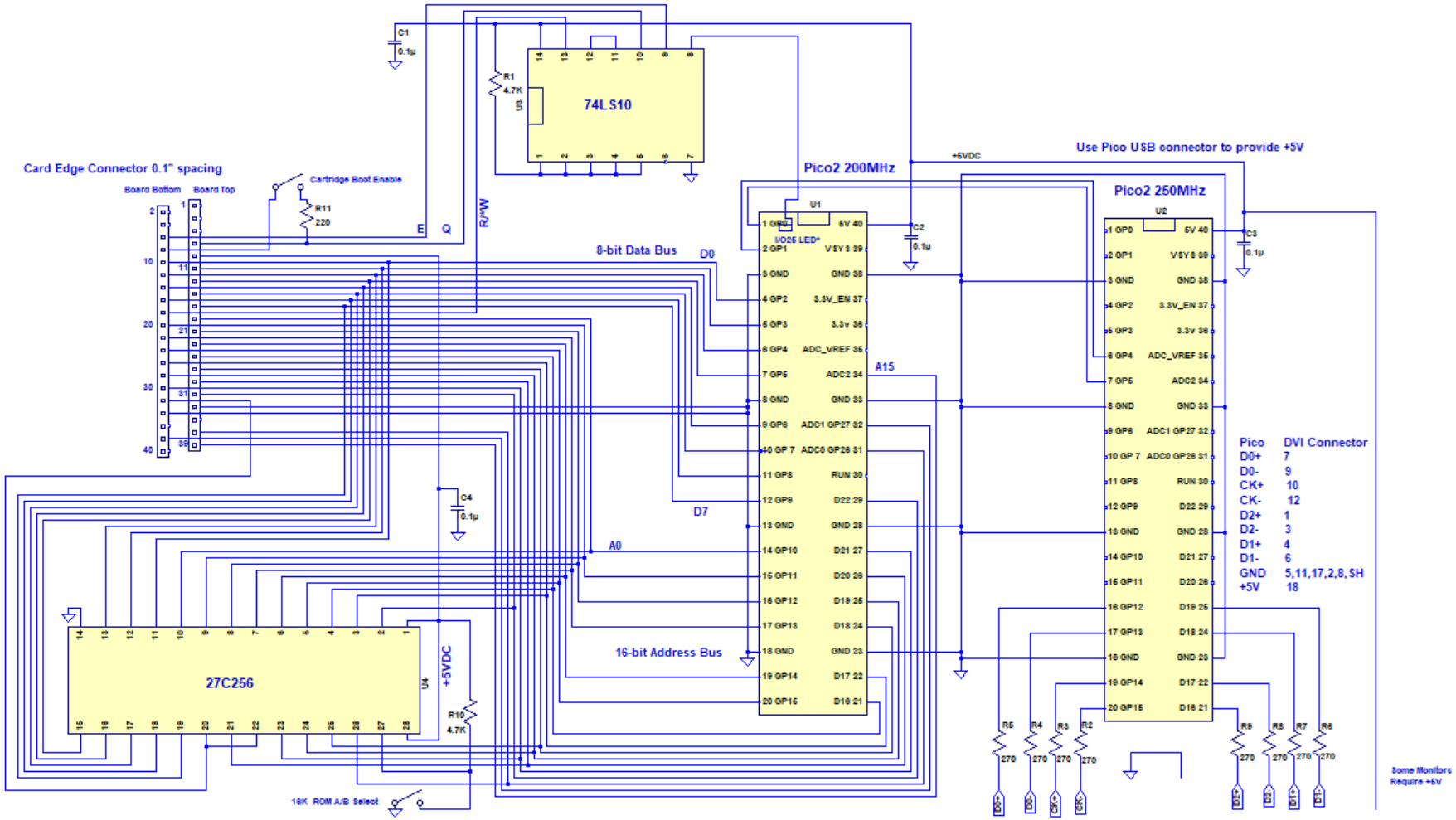
The DVI signal is generated using the Raspberry Pi HSTX DVI example code. A software modification was made to the clock divider to allow the Pico to run overclocked at 250MHz. The bus watching Pico records all memory writes including writes to the SAM chip and the PIA chips. Writes to these devices are used to identify the video RAM location and the video modes. The mode and video memory are sent to the video Pico after it receives a serial byte. If no byte is received after 50ms, a time out occurs and data is sent. The bus watching Pico is overclocked at 200MHz.

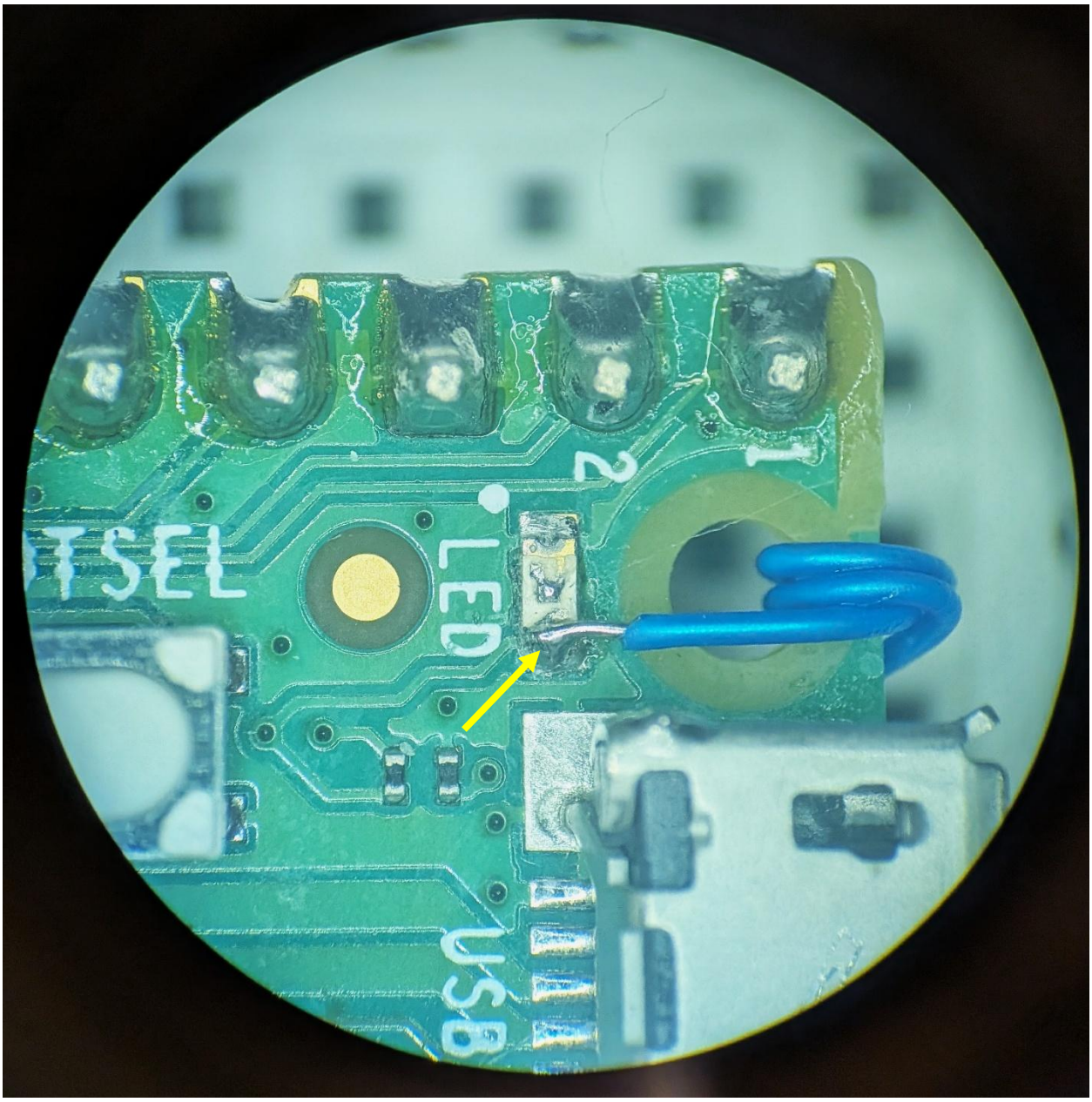
The initial build was made using supplies on hand. It was wired using stripped, hooked and soldered 30 gage wire wrap wire. After operation of the video interface was verified, a 27C256 EPROM socket was added.

The color computer does not provide enough current for the video interface. The EPROM is powered from the cartridge bus, but the two Pico processors are powered from a separate 5VDC source using one of the Pico USB connectors. The USB power turns on first, then the Color Computer is powered on. I have performed the sequence incorrectly many times with no failures, but I wouldn't guarantee that CMOS latch-up and damage to the processor won't occur with the reverse power sequence.

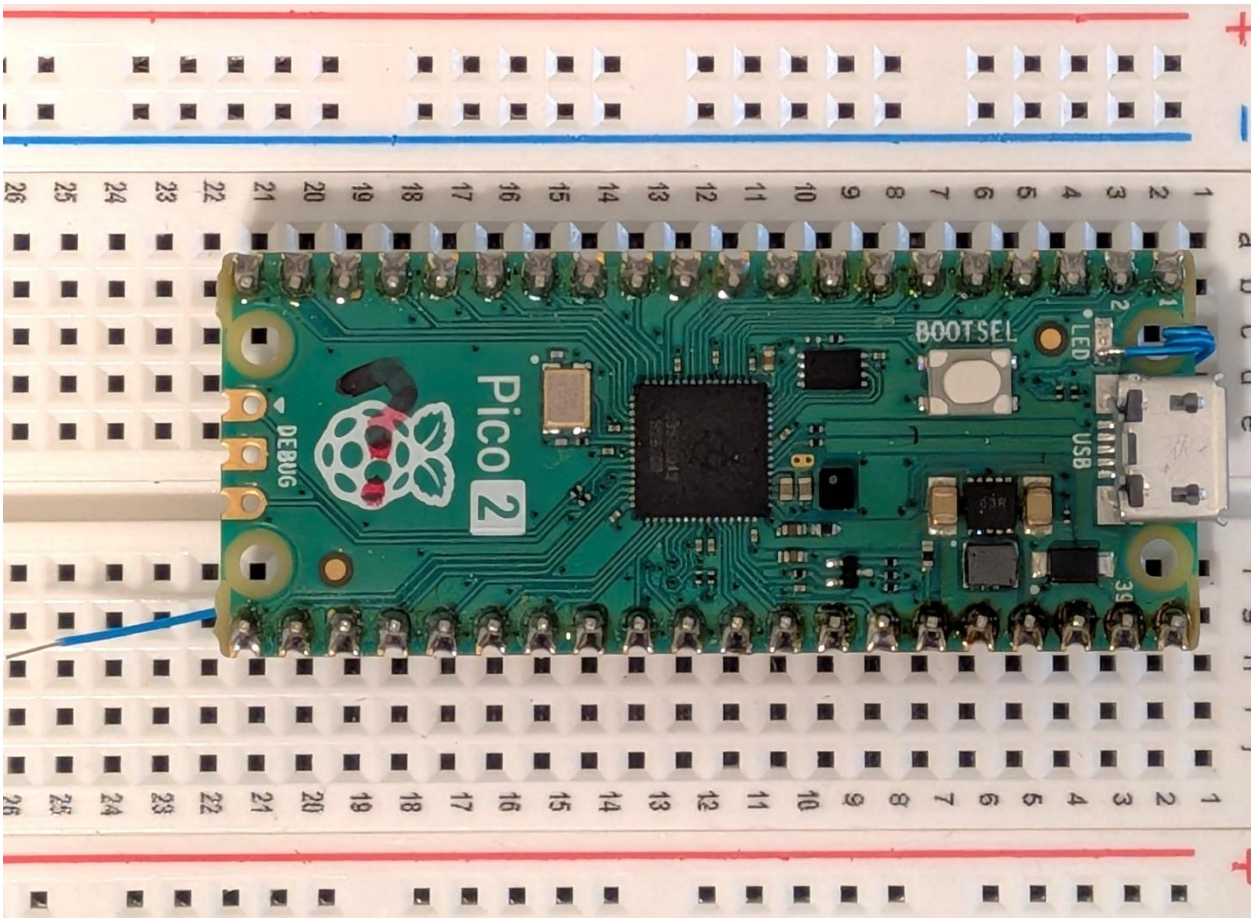
A schematic diagram of the circuit is given on the next page. Some build tips are also included. I also made circuit card using KiCad. Machine tooled 0.1" socket strips are used for the dual-in-line 40-pin, 28-pin and 14-pin devices. These socket strips give extra board height clearances.

In Kicad, the edge card fingers were generated with fat lines and connected to a 20x2 socket strip library part. The silk screen for the 20x2 socket is problematic. To prevent printing over the fingers from the socket strip, the silk screen layer is not used for the board production. The card is quite simple and part locations are show in this document.

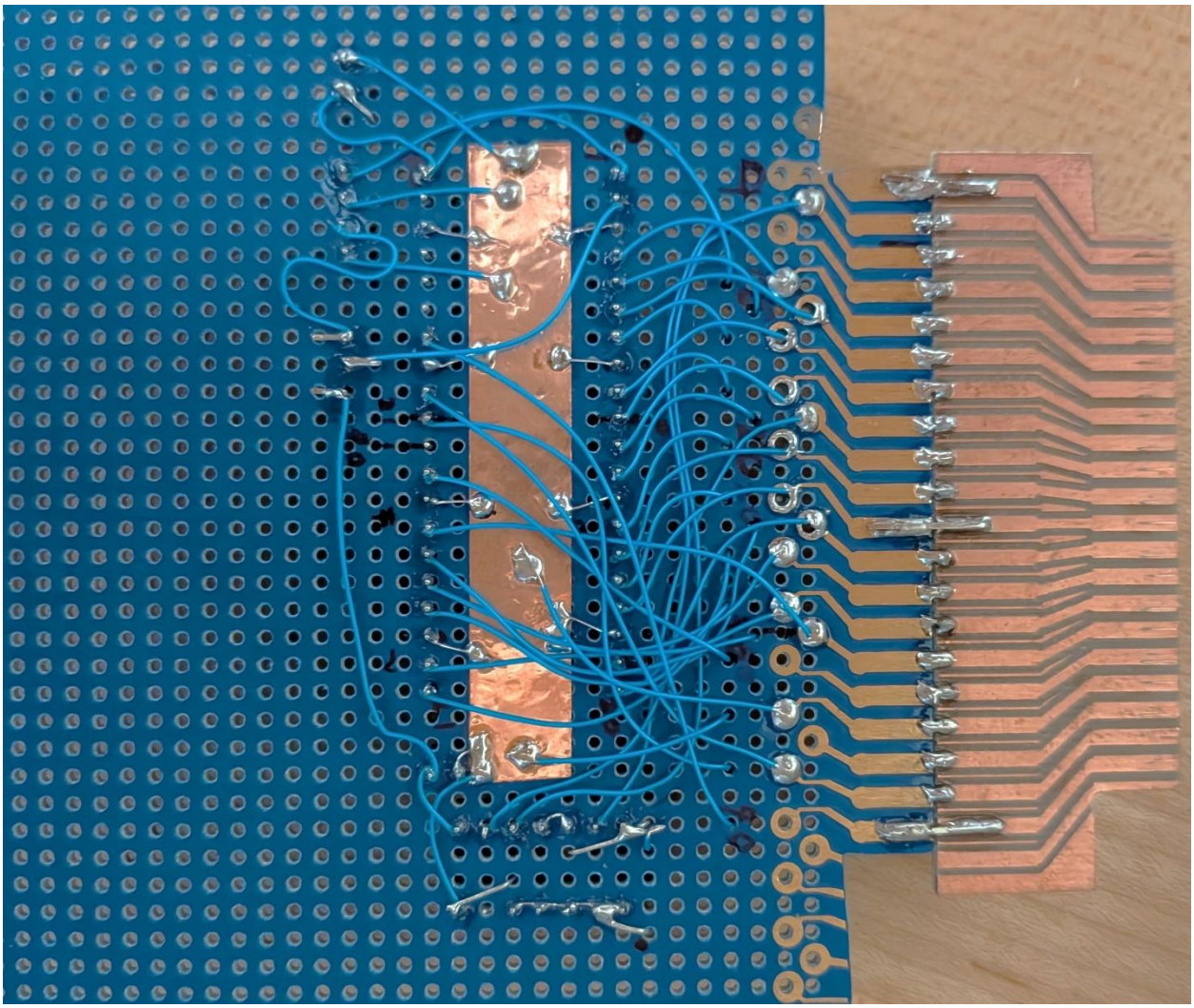

Card Edge Connector 0.1" spacing
Board Bottom
Board Top
Cartridge Boot Enable
R11
220
E
Q
R/W
C1
0.1µ
R1
4.7K
U3
74LS10
Pico2 200MHz
+5VDC
Use Pico USB connector to provide +5V
Pico2 250MHz
U1
U2
I/O25 LED*
C2
0.1µ
C3
0.1µ
8-bit Data Bus
D0
D7
A15
A0
16-bit Address Bus
C4
0.1µ
27C256
U4
+5VDC
R10
4.7K
16K ROM A/B Select
R5
R4
R3
R2
R9
R8
R7
R6
270
Pico DVI Connector
D0+ 7
D0- 9
CK+ 10
CK- 12
D2+ 1
D2- 3
D1+ 4
D1- 6
GND 5,11,17,2,8, SH
+5V 18
Some Monitors Require +5V



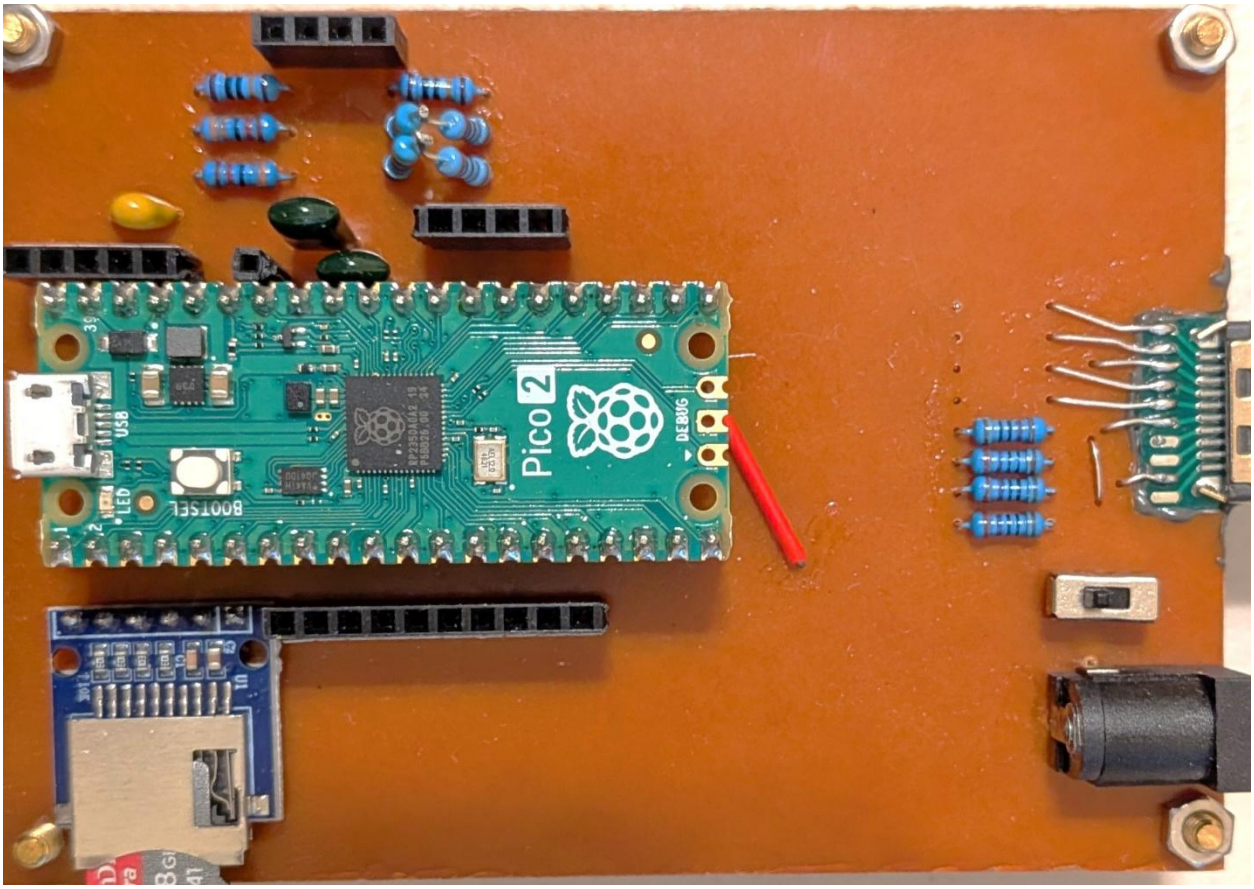
The clear plastic to the LED was rubbed with a hot soldering iron tip util it decomposed. Continued rubbing removed the die and wire bond. The inside edge was wet with solder and a 30 gage wire wrap wire was soldered to it. This is the I/O-25 input. The wire was strain relieved by wrapping a few turns in the hole.



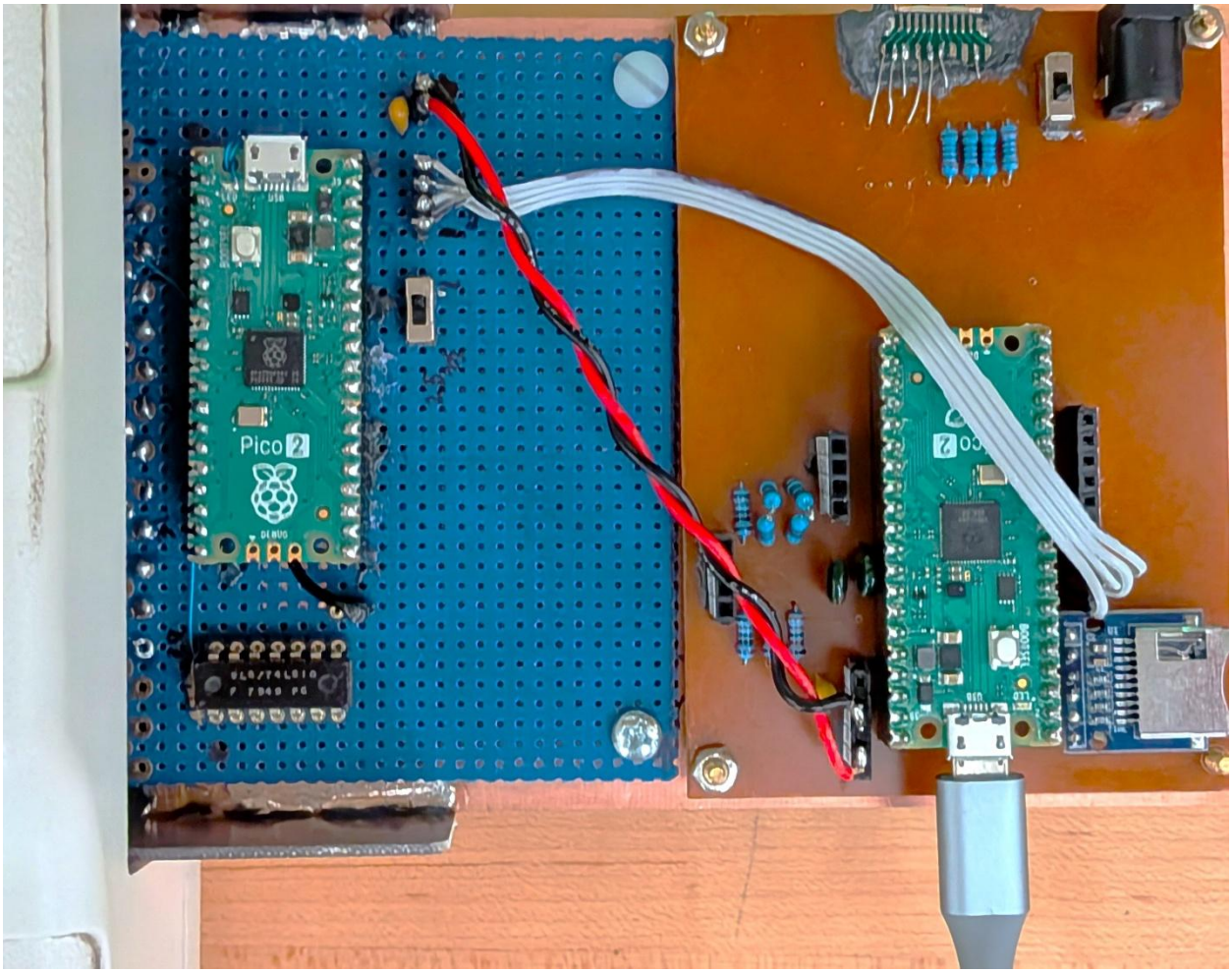
The completed I/O-25 wire connection. This processor was cooked when I tried to run at 300MHz. It still operated but drew higher than normal current.



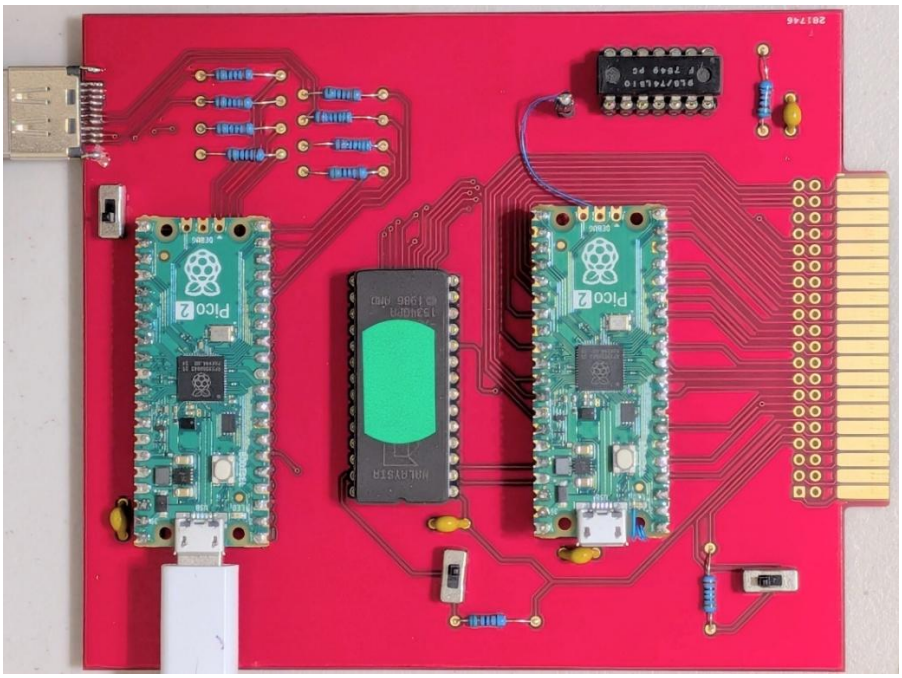
The prototype board I had on hand had an edge connector with 1/8” spaced fingers. I milled an adapter board for the correct 0.1” finger spacing. This board has all the wiring for the bus capture. The EPROM was wired and added later.



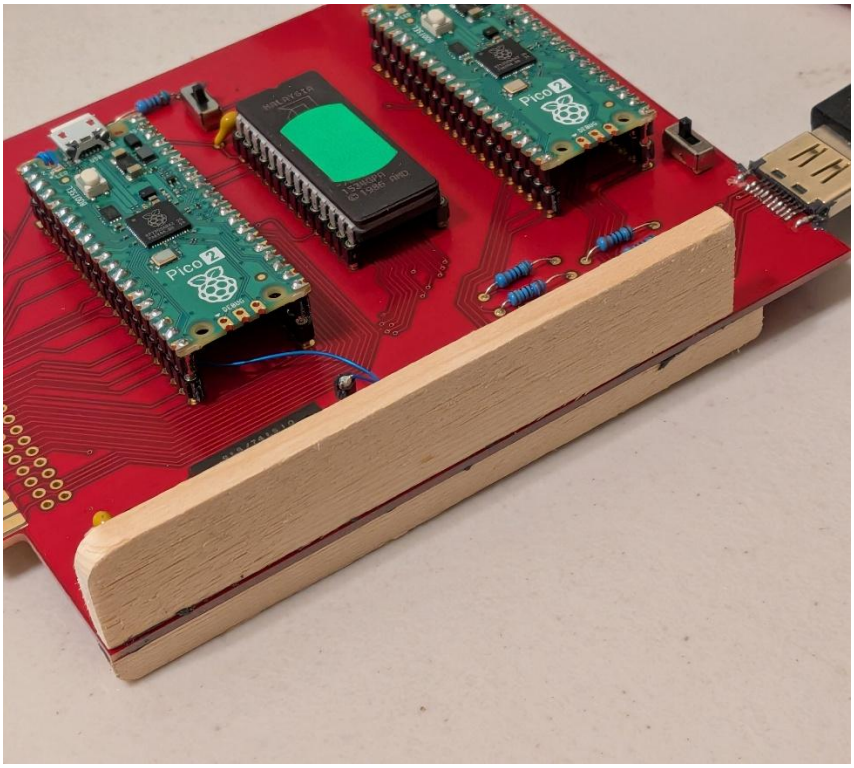
Being lazy, I reused a board I made for a PicoMite project. This is a CNC milled board.



The two boards were connected to a double-sided PCB base. The base is 4-1/8” wide by 7” long. The bus capture board fingers extend ¼” in front of the base edge and the capture board is 1/8” above the base. The base board is 0.062” thick as is the edge connector on the capture board. The base sits flush on the cartridge opening. The sides are fitted with 7/8” x 3” board pieces to act as door openers.



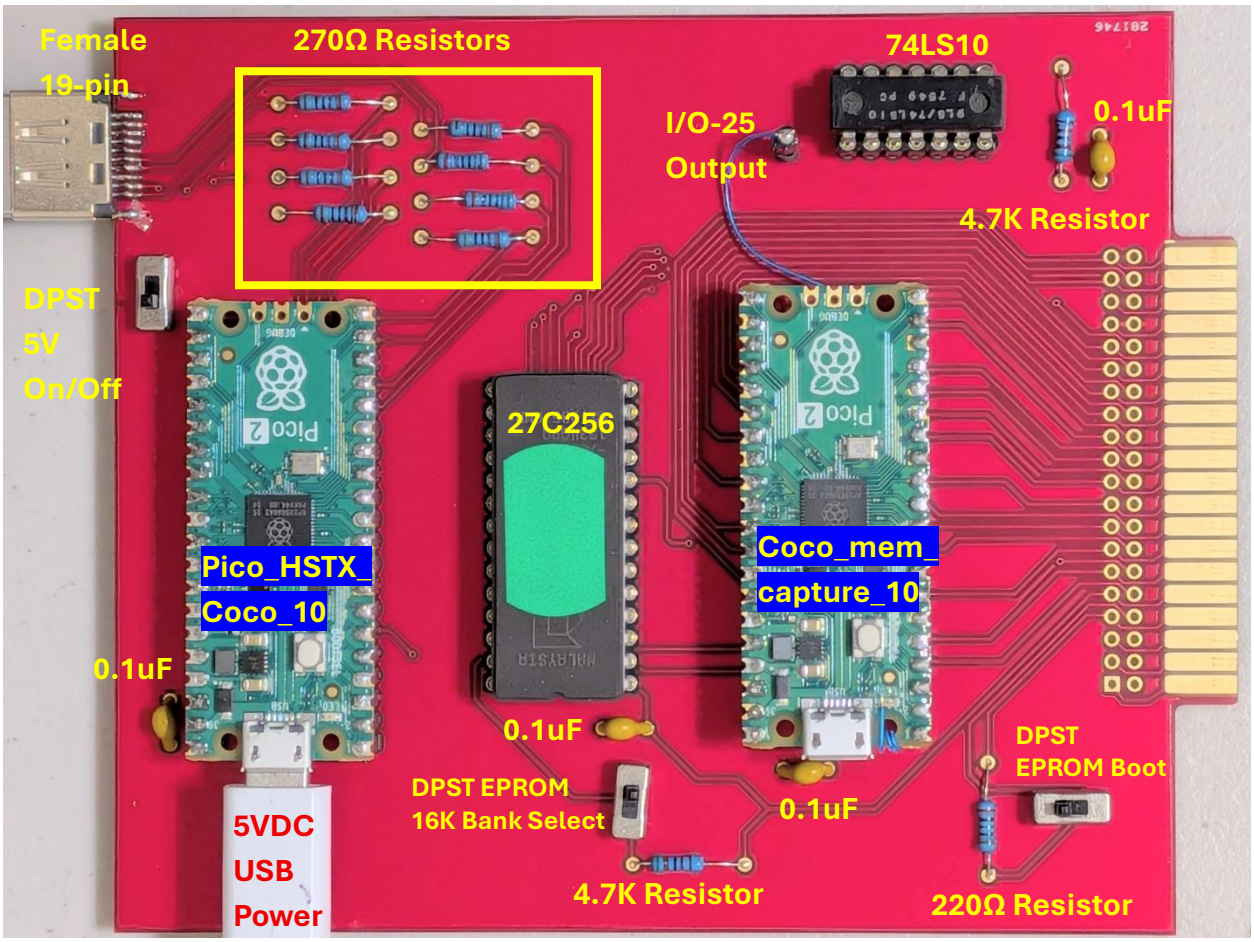
The two-layer PCB is a bit easier to build than the hand wired board. Gerber and drill files are available.



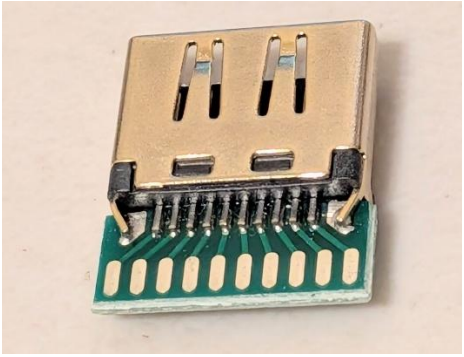
I don’t have a 3-D printer to generate a plastic case. I epoxied two 3/8” tall by 1/8” wide wood strips to the bottom edges of the card and a 5/8” tall by 1/8” wide strip to the top edge opposite of the Pico USB input connectors. This forms the door opener.

PCB Build Parts List:

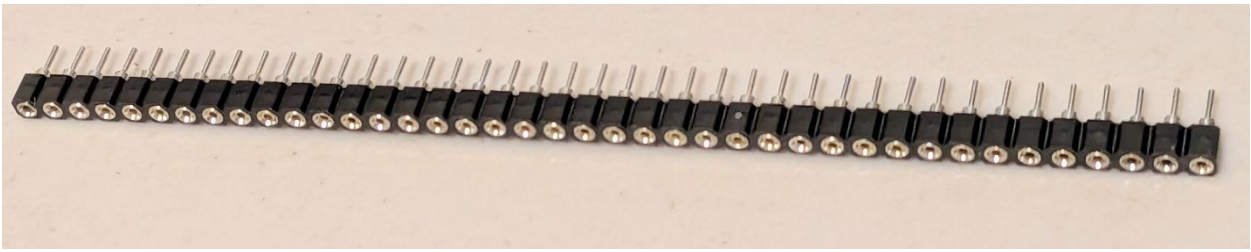
2 – Raspberry Pi Pico 2 processors
8 – 40-pin machine tooled 0.1” spaced socket strips
40 – Inches of 26-gage solid wire
1 – 74LS10 Triple 3-Input NAND gate
1 – Optional 27C256 UV erasable EPROM
    (programmed with one or two of your 16Kx8 ROM images)
3 – DPST slide switches with 0.1” spaced leads
8 – 270Ω 1/4W metal or carbon film resistors
2 – 4.7K or 5.1K 1/4W metal or carbon film resistors
1 - 220Ω 1/4W metal or carbon film resistor
4 – 0.1uF ceramic capacitors with 0.2” lead spacing
4 – Inches of 30-gage wire wrap wire or equivalent small wire
2 – 1/8” wide by 3/16” tall by 4” long wood strips
1 – 1/8” wide by 5/8” tall by 4” long wood strip
1 – Video Female Jack/Plug Connector 19PIN 19P 1.6MM 180 Degree
5-minute epoxy to glue door opener strips
Eutectic solder (I used 63/37 tin/lead with a rosin core)



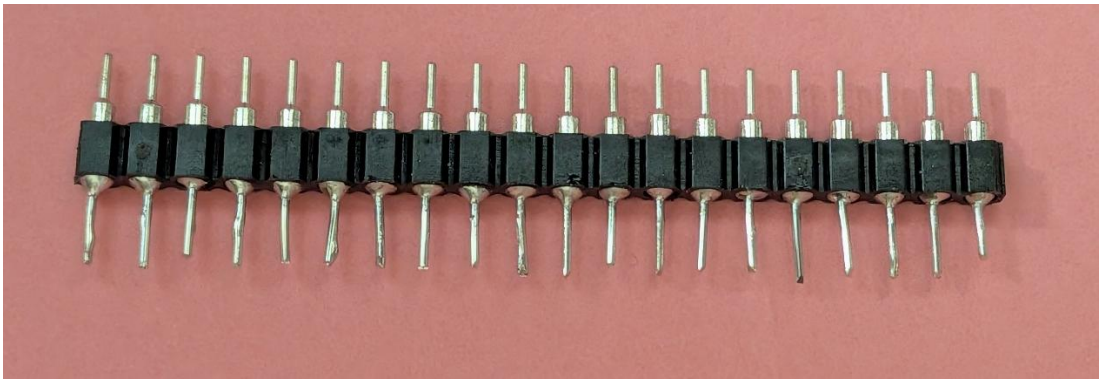
Component locations on the PCB. The wide part of the video connector is up towards the top component layer.



The video connector has two rows of pins for soldering to the top and bottom of the board edge. Amazon does list a female connector without the small PC board. Initially, I didn't have one on hand and I removed the board using solder wick. If you go this route, be careful as the pins pull out of the connector very easily.



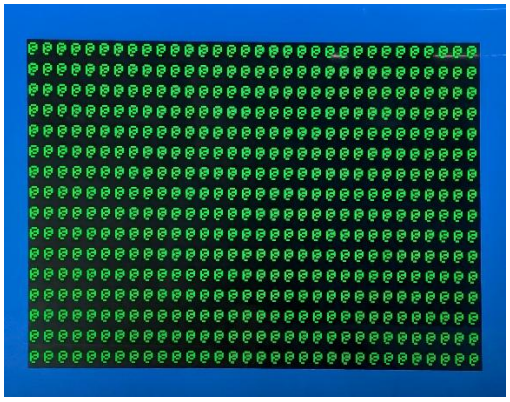
The board was socketed with machine tooled pin strips. This was to keep a low height profile on the Pico processors. Sockets make replacement easier should something fail.



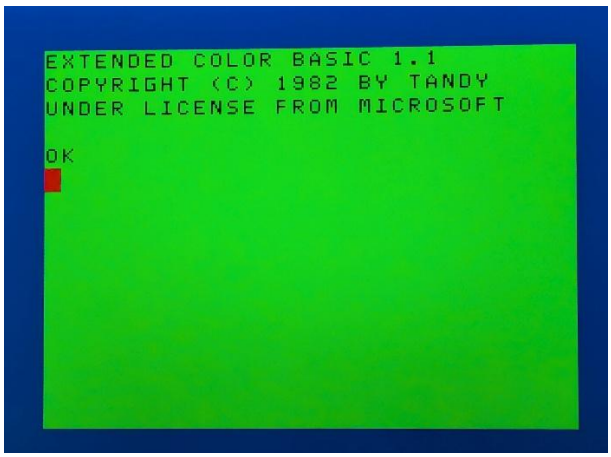
½" 26-gage wires were inserted into the socket strips to form the male pin headers for the Pico processors. The wires are soldered into the sockets before soldering to the Pico. Solder the female strips into the boards first. Slip the 26-gage wires into the Pico solder pad holes. Insert the pins into the board connectors then solder the wires to the Pico. This ensures the male pins will align with the sockets.

After all components are installed, plug a USB cable from your computer into the Pico you want to program. The UF2 files are: 1) **coco_mem_capture_11.ino.rpipico2.uf2** and 2) **pico_hstx_coco_11.ino.rpipico2.uf2**

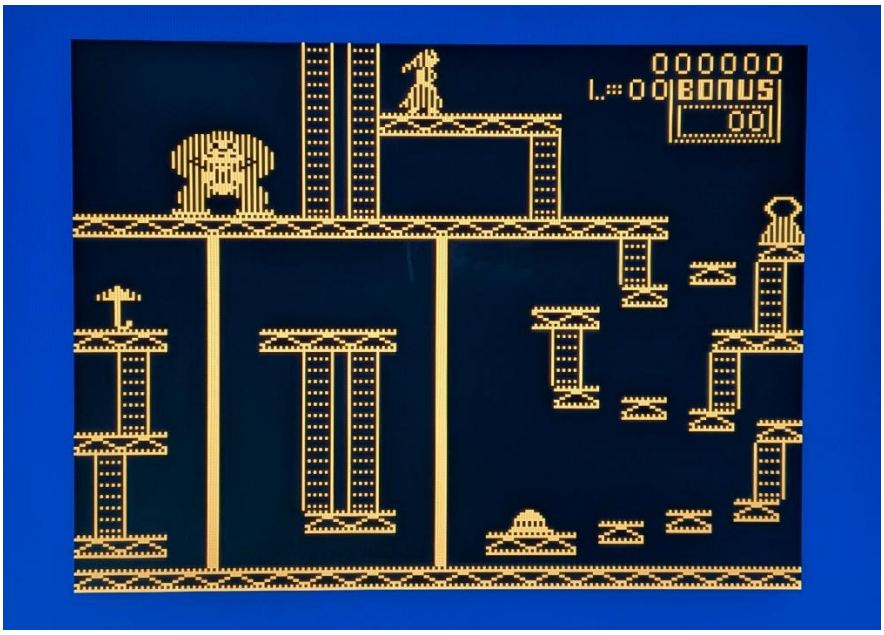
After programming both Pico processors, power the pico_hstx processor with 5V through a USB cable. Plug in the video cable to the DVI monitor. The card boots with zeros in the text memory. This displays '@' characters.



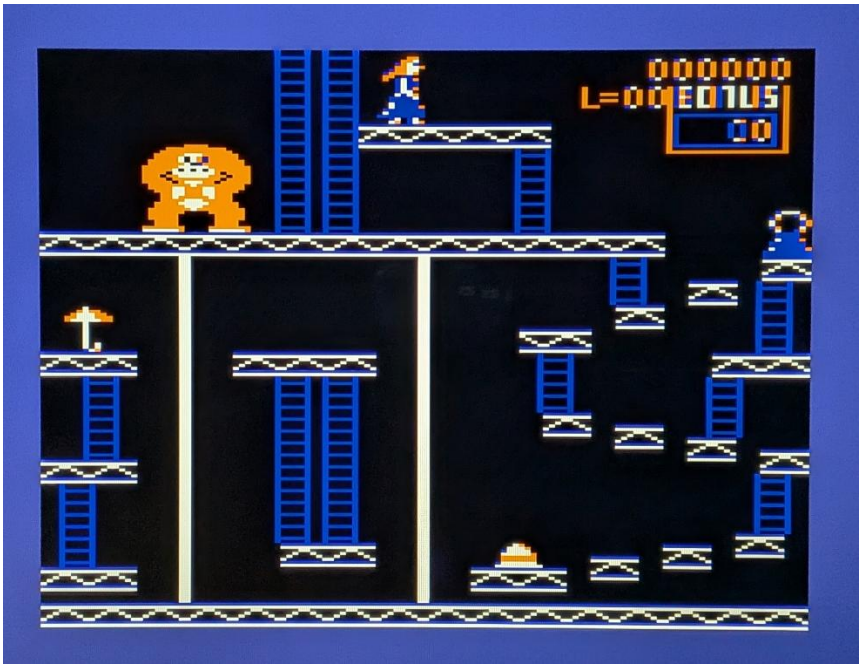
Power off the board, insert it into your Color Computer II. Power the board on first, then power the CoCo up. The video portion of the board appears only as inputs to the Cartridge bus. The EPROM is powered by the CoCo. Powering the board up first ensures, the Pico capturing the memory writes, does not get powered up through the I/O pins.



Power down in the reverse order. Turn the CoCo off first, then power down the video board.



The 256x192 mode is used on many games. They rely on NTSC artifact colors and they will look grated.



An artifact feature was added for the 256x192 mode. It is turned on by shorting pin-2 to pin-3 (ground) on the video generating Pico. It is experimental but simple. It forces the 256x192 mode to use the 128x192 four color mode and sets the palette to black, blue, orange, white. Color is only correct for pixels starting on even bits.

Resources:

https://github.com/raspberrypi/pico-examples/tree/master/hstx/dvi_out_hstx_encoder

https://colorcomputerarchive.com/

https://geoffg.net/picomite.html

https://www.digikey.com/

https://www.jameco.com/shop/KeywordSearchResultView?langId=-1&storeId=10001&catalogId=10001&search_type_c=jamecoall&freeText=eprom

https://github.com/earlephilhower/arduino-pico

https://www.arduino.cc/en/software/
(I use IDE 1.8.12)

Amazon parts search for:

Video Connector:
5PCS HDMI Male/Plug C0innector 19PIN 19P 180 DeGrrie Female Plug Female Socket USpB HD C0innector SMT 90 DeGrrie PCB

Socket Strips:
Single Row Round Headers Machine Pin Female 0.1" 2.54mm 40 Pins(Pack of 10)

https://github.com/simple-circuit/cocodvi

To compile pico_hstx_coco_11.ino, use the following Arduino-pico settings:

```
Board: "Raspberry Pi Pico 2"
Flash Size: "4MB (no FS)"
CPU Speed: "250 MHz (Overclock)"
CPU Architecture: "ARM"
Optimize: "Optimize (-O)"
Profiling: "Disabled"
RTTI: "Disabled"
Stack Protector: "Disabled"
C++ Exceptions: "Disabled"
Debug Port: "Disabled"
Debug Level: "None"
USB Stack: "No USB"
IP/Bluetooth Stack: "IPv4 Only"
Upload Method: "Default (UF2)"
```

To compile coco_mem_capture_11.ino, use the following Arduino-pico settings:

```
Board: "Raspberry Pi Pico 2"
Flash Size: "4MB (no FS)"
CPU Speed: "200 MHz (Overclock)"
CPU Architecture: "ARM"
Optimize: "Optimize (-O)"
Profiling: "Disabled"
RTTI: "Disabled"
Stack Protector: "Disabled"
C++ Exceptions: "Disabled"
Debug Port: "Disabled"
Debug Level: "None"
USB Stack: "No USB"
IP/Bluetooth Stack: "IPv4 Only"
Upload Method: "Default (UF2)"
```